Andrew Nguyen
4420 HW2

Environment used: Blackjack

A]
Goal: Get hand as close to 21 without going over while being closer than the dealer
State Space: (Player Hand Value, Dealer's Card, Usable Ace (1/11))
Action Space: 0 = Stand, 1 = Hit
Transition Dynamics: If 0 then the State Space remains the same, if 1 then the Player Hand Value will increase by 1-10
Reward Structure: +1 if win, -1 if lose, 0 if draw
Termination: When the hand is completed (win, lost, drawn)

B] (taken from the documentation)

```
    """Update Q-value based on experience.

    This is the heart of Q-learning: learn from (state, action, reward, next_state)
    """
    # What's the best we could do from the next state?
    # (Zero if episode terminated - no future rewards possible)
    future_q_value = (not terminated) * np.max(self.q_values[next_obs])

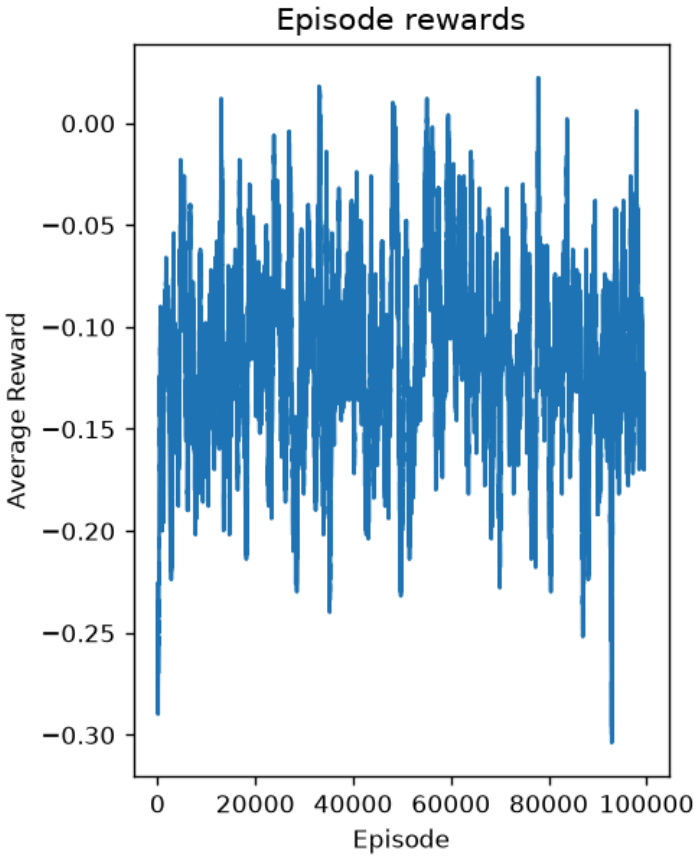
    # What should the Q-value be? (Bellman equation)
    target = reward + self.discount_factor * future_q_value

    # How wrong was our current estimate?
    temporal_difference = target - self.q_values[obs][action]

    # Update our estimate in the direction of the error
    # Learning rate controls how big steps we take
    self.q_values[obs][action] = (
        self.q_values[obs][action] + self.lr * temporal_difference
    )
```

C] parameters

```
#CHNAGE STUFF HERE
alpha = 0.5 # learning rate
gamma = 0.9 # discount factor
epsilon = 0.05 # exploration factor
n_episodes = 5000
```



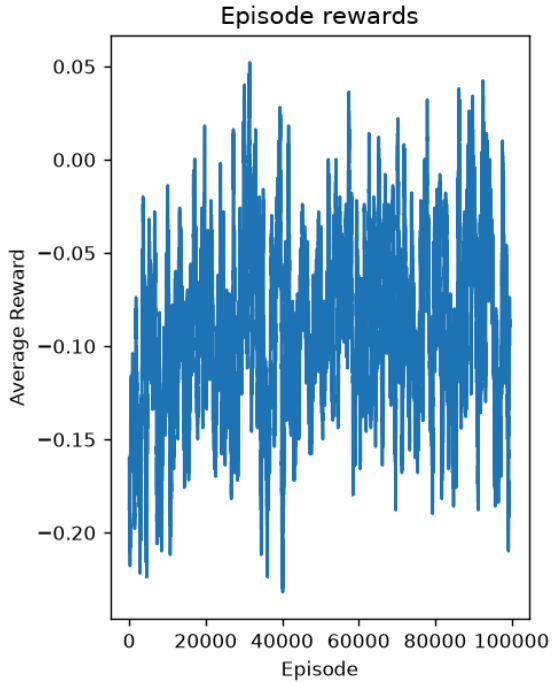

```
Win Rate: 40.7%
Average Reward: -0.105
Standard Deviation: 0.953
```

For future purposes, according to the Gymnasium Documentation page (https://gymnasium.farama.org/introduction/train_agent/) a 42-45% win rate and an average reward of -0.02 to +0.01 is considered good.

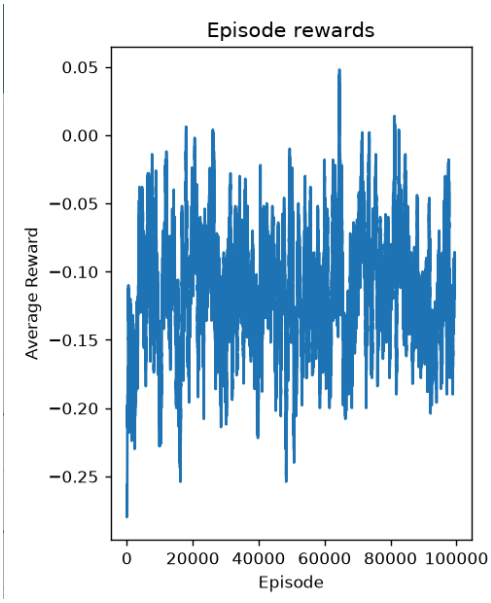
This baseline training seems to show that the agent performs slightly below average and also had not found a consistent strategy due to the high standard deviation compared to the average.

D]
Testing Values at
alpha = 0.01, 0.5 (baseline), 0.9

alpha=0.01 (Win Rate = 42.0%, Avg Reward = -0.062, Std. dev. = 0.948)



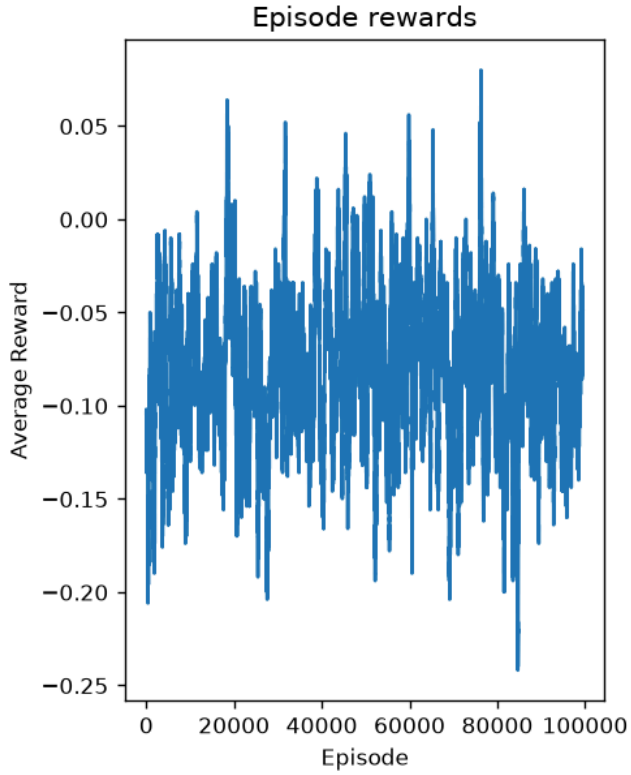
alpha=0.5 (use baseline value)
alpha = 0.9 (Win Rate = 42.6%, Avg Reward = -0.058, Std. dev. = 0.952)



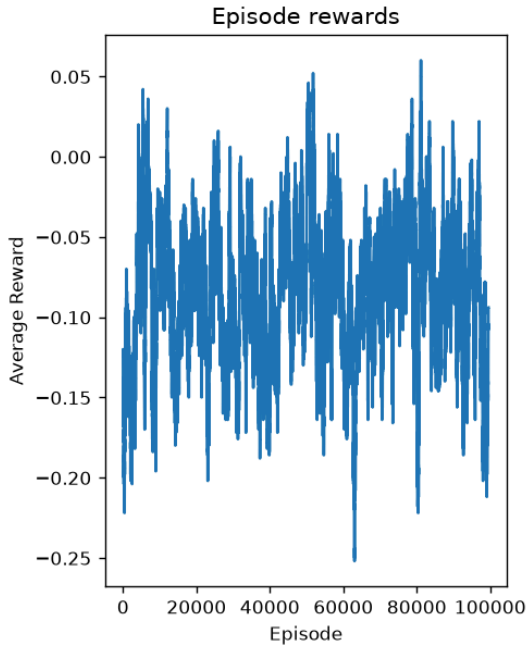
It seems like the difference between changing alpha values does not greatly affect anything as they perform relatively similarly with the same instability.

E]
Testing values of gamma at 0.1, 0.5, 0.99

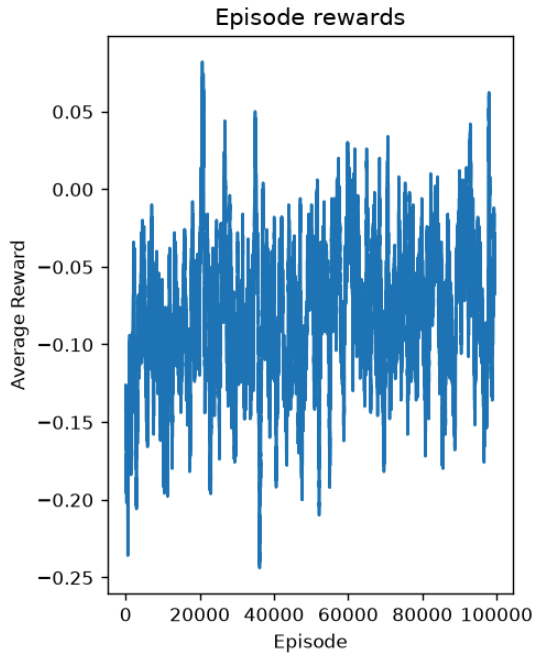
gamma = 0.1 (Win Rate = 44.1%, Avg Reward = -0.013, Std. dev. = 0.946)



gamma = 0.5 (Win Rate = 43.4%, Avg Reward = -0.035, Std. dev. = 0.950)

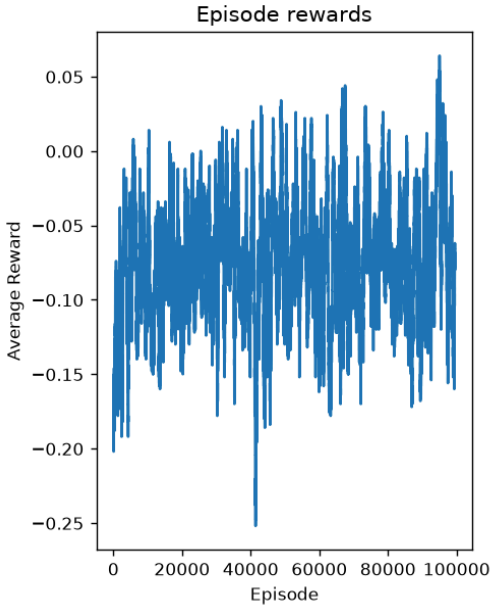


gamma=0.99 (Win Rate = 42.8%, Avg Reward = -0.042, Std. dev. = 0.947)

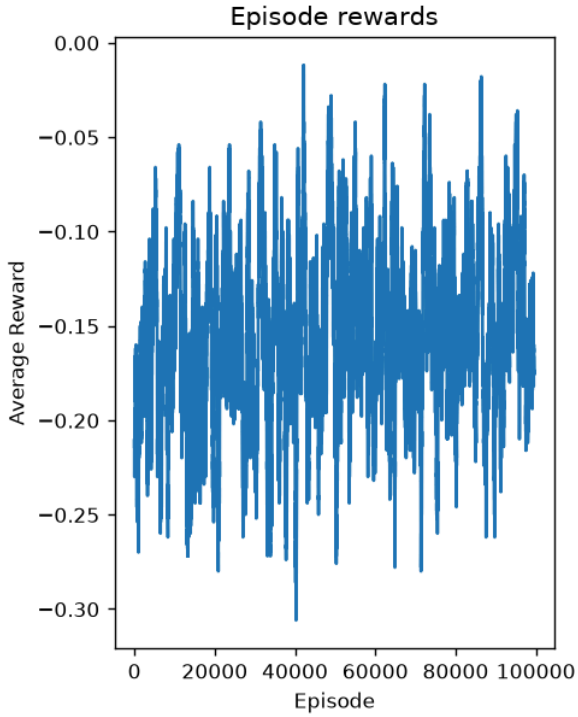


While still unstable, it seems like lower gamma leads to higher win rates overall despite the lower gammas leading to more fluctuation in the graph.

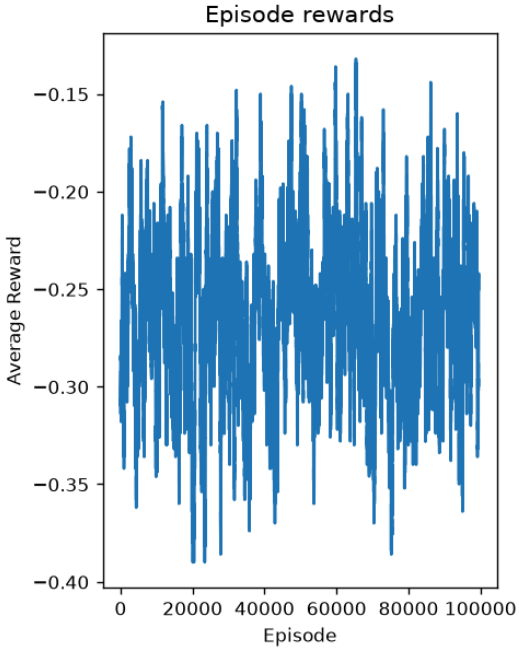
F]
Testing values of epsilon at 0.01, 0.25, 0.6
epsilon = 0.01 (Win Rate = 42.8%, Avg Reward = -0.042, Std. dev. = 0.947)



epsilon = 0.25 (Win Rate = 40.4%, Avg Reward = -0.103, Std. dev. = 0.949)



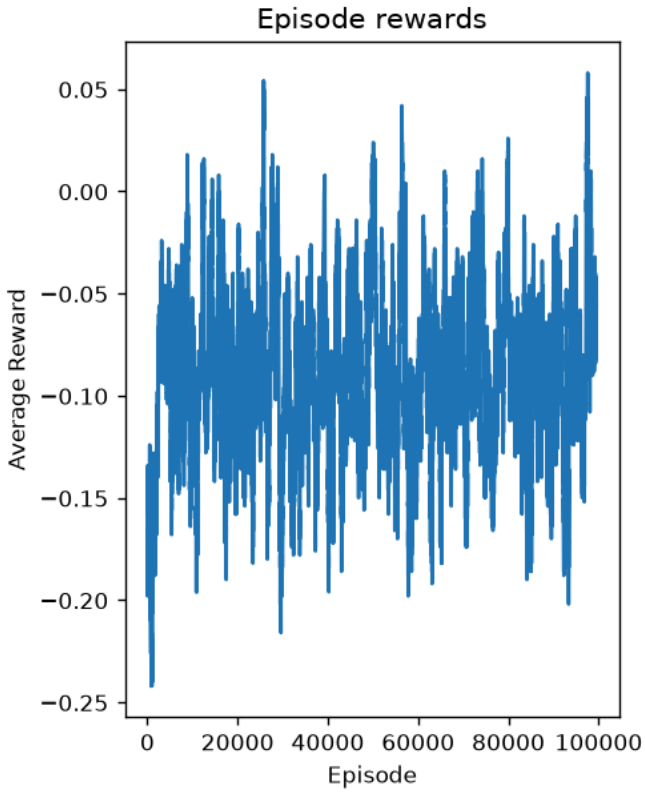
epsilon = 0.6 (Win Rate = 42.2%, Avg Reward = -0.071, Std. dev. = 0.954)



It seems like higher epsilon values lead to more fluctuation in rewards and lower win rates compared to higher epsilon.

G]
Testing with these parameters

```
alpha = 0.5 # learning rate
gamma = 0.05 # discount factor
epsilon = 0.0 # exploration factor
n_episodes = 100000
```




```
Win Rate: 40.0%
Average Reward: -0.086
Standard Deviation: 0.937
```

I chose a middle ground in learning rate as observations did not change much about it. I chose a low gamma as observations showed that lower gamma lead to higher win rates and epsilon is at 0 because exploration is disabled. This leads to a poor result as the model is not allowed to explore different strategies.

Conclusion: Blackjack is designed in such a way that with optimal play, the dealer will always have an edge, which is why the win rate is below 50% and the average reward tends to be on the more negative side. Additionally, due to the randomness of blackjack, stable strategies will be difficult to find as the game at the end of the day is gambling.